

Amelia Serrano Arango – 202221556

Santiago Baquero Villa- 202220396

Mateo Jaimes Rincón - 202411187

María Alejandra Sanabria Salazar- 202326786

ISIS – 1611: Inteligencia Artificial

Taller 4: Planificación Automatizada

1. Modelado del Dominio de Planificación

a. Complejidad en espacio (Notación O).

Las funciones implementadas en el Punto 1 trabajan directamente sobre estados representados como conjuntos de flujos y sobre acciones instanciadas, la función `is_applicable` únicamente revisa las precondiciones de una acción y no necesita memoria adicional significativa, por lo que su complejidad espacial es $O(1)$, la función `apply_action` crea un nuevo estado copiando y modificando el estado actual, por lo que requiere $O(n)$ memoria, donde n es el número de flujos del estado y finalmente, `get_applicable_actions` puede almacenar todas las acciones instanciadas aplicables en el estado actual, lo cual requiere $O(a)$ memoria, donde a representa el número de acciones generadas.

b. Complejidad en tiempo (Notación O).

La complejidad temporal de `is_applicable` depende del número de precondiciones de la acción, ya que debe verificar cada una, por lo que es $O(p)$, la función `apply_action` debe recorrer los flujos del estado para eliminar y agregar efectos, por lo que su complejidad es $O(n)$ y por otro lado, `get_applicable_actions` es la función más costosa del punto, porque genera todas las acciones posibles y verifica su aplicabilidad, por ende, su complejidad aproximada es $O(a \cdot p)$, donde a es el número de acciones instanciadas y p el número de precondiciones promedio.

c. Completitud

Las funciones del Punto 1 no son algoritmos de búsqueda completos por sí mismas, ya que únicamente representan y manipulan estados y acciones del dominio, sin embargo, son indispensables para que los algoritmos de planificación posteriores puedan explorar correctamente el espacio de estados y determinar qué acciones son válidas en cada situación.

d. Optimalidad

El Punto 1 no implementa ningún algoritmo de búsqueda ni de optimización, por lo que no garantiza encontrar planes óptimos. La optimalidad depende completamente del algoritmo de planificación utilizado posteriormente, como BFS, A^* o HTN.

e. Reflexión de la co-construcción con la IA

Durante el desarrollo del punto 1 no se utilizó inteligencia artificial para generar directamente la solución del código ni para resolver automáticamente el problema de planificación clásica, la implementación se realizó completando las funciones propuestas en el taller, siguiendo la lógica y la estructura ya definida en los archivos base. Sin embargo, sí se utilizó apoyo de herramientas de IA dentro de Visual Studio Code para mejorar la estructura de escritura del código.

2. Búsqueda hacia Adelante

a. Análisis del comportamiento de forwardBFS.

El algoritmo Forward BFS encontró soluciones óptimas en ambos layouts, pero con un costo computacional alto. En mediumRescue, encontró un plan de 32 acciones, expandiendo 5,148 estados en 13.02 segundos. En warehouseRescue, encontró un plan ligeramente más corto, de 31 acciones, pero expandió 9,716 estados y tardó 57.27 segundos. Esto muestra que la longitud del plan no determina directamente el esfuerzo computacional. Aunque warehouseRescue produjo una solución más corta, requirió casi el doble de estados explorados. Esto ocurre porque BFS explora todos los estados nivel por nivel, sin usar información heurística para priorizar caminos prometedores. Por tanto, la estructura del entorno influye significativamente, los layouts con más rutas alternativas o mayor ramificación incrementan el tamaño del espacio de búsqueda, incluso si la solución final es similar en longitud. En conclusión, BFS garantiza planes óptimos en número de acciones, pero su rendimiento disminuye considerablemente en entornos más complejos debido al crecimiento del número de estados explorados.

```

(base) ameliasserrano@air-de-amelia-2 PhoenixOperation % python3 main.
=====
Operación Fénix - ISIS-1611 Inteligencia Artificial
=====
Layout: mediumRescue (12x12)
Problema: SimpleRescueProblem
Pacientes: ['patient_0']
Suministros: ['supplies_0']
Puestos médicos: [(4, 2)]

Planificador: forwardBFS

Tiempo de planificación: 13.020s
Estados expandidos: 5148
Longitud del plan: 32 acciones

Plan:
1. Move(robot, (1, 10), (1, 9))
2. Move(robot, (1, 9), (1, 8))
3. Move(robot, (1, 8), (2, 8))
4. Move(robot, (2, 8), (3, 8))
5. Move(robot, (3, 8), (4, 8))
6. Move(robot, (4, 8), (5, 8))
7. Move(robot, (5, 8), (5, 7))
8. Move(robot, (5, 7), (5, 6))
9. Move(robot, (5, 6), (4, 6))
10. Pickup(robot, supplies_0, (4, 6))
11. Move(robot, (4, 6), (5, 6))
12. Move(robot, (5, 6), (5, 7))
13. Move(robot, (5, 7), (5, 8))
14. Move(robot, (5, 8), (4, 8))
15. Move(robot, (4, 8), (3, 8))
16. Move(robot, (3, 8), (2, 8))
17. Move(robot, (2, 8), (1, 8))
18. Move(robot, (1, 8), (1, 7))
19. Move(robot, (1, 7), (1, 6))
20. Move(robot, (1, 6), (1, 5))
21. Move(robot, (1, 5), (1, 4))
22. Move(robot, (1, 4), (1, 3))
23. Move(robot, (1, 3), (1, 2))
24. Move(robot, (1, 2), (2, 2))
25. Move(robot, (2, 2), (3, 2))
26. Move(robot, (3, 2), (4, 2))
27. SetupSupplies(robot, supplies_0, (4, 2))
28. Move(robot, (4, 2), (4, 1))
29. Pickup(robot, patient_0, (4, 1))
30. Move(robot, (4, 1), (4, 2))
31. PutDown(robot, patient_0, (4, 2))
32. Rescue(robot, patient_0, (4, 2))

Ejecutando plan...
¡Misión completada exitosamente!

```

```

=====
Operación Fénix - ISIS-1611 Inteligencia Artificial
=====
Layout: warehouseRescue (15x12)
Problema: SimpleRescueProblem
Pacientes: ['patient_0']
Suministros: ['supplies_0']
Puestos médicos: [(3, 2)]

Planificador: forwardBFS

Tiempo de planificación: 57.269s
Estados expandidos: 9716
Longitud del plan: 31 acciones

Plan:
1. Move(robot, (1, 10), (2, 10))
2. Move(robot, (2, 10), (3, 10))
3. Move(robot, (3, 10), (4, 10))
4. Move(robot, (4, 10), (5, 10))
5. Move(robot, (5, 10), (6, 10))
6. Move(robot, (6, 10), (7, 10))
7. Move(robot, (7, 10), (7, 9))
8. Move(robot, (7, 9), (7, 8))
9. Move(robot, (7, 8), (7, 7))
10. Pickup(robot, supplies_0, (7, 7))
11. Move(robot, (7, 7), (7, 8))
12. Move(robot, (7, 8), (7, 9))
13. Move(robot, (7, 9), (7, 10))
14. Move(robot, (7, 10), (6, 10))
15. Move(robot, (6, 10), (5, 10))
16. Move(robot, (5, 10), (4, 10))
17. Move(robot, (4, 10), (3, 10))
18. Move(robot, (3, 10), (3, 9))
19. Move(robot, (3, 9), (3, 8))
20. Move(robot, (3, 8), (3, 7))
21. Move(robot, (3, 7), (3, 6))
22. Move(robot, (3, 6), (3, 5))
23. Move(robot, (3, 5), (3, 4))
24. Move(robot, (3, 4), (3, 3))
25. Move(robot, (3, 3), (3, 2))
26. SetupSupplies(robot, supplies_0, (3, 2))
27. Move(robot, (3, 2), (3, 1))
28. Pickup(robot, patient_0, (3, 1))
29. Move(robot, (3, 1), (3, 2))
30. PutDown(robot, patient_0, (3, 2))
31. Rescue(robot, patient_0, (3, 2))

Ejecutando plan...
¡Misión completada exitosamente!

```

b. Complejidad en espacio (Notación O).

La complejidad en espacio de BFS es $O(b^d)$, donde b es el factor de ramificación (número promedio de sucesores por estado) y d la profundidad de la solución. Esto se debe a que BFS debe almacenar en memoria todos los nodos generados en la frontera y los estados visitados, lo que provoca un alto consumo de memoria a medida que crece el espacio de búsqueda.

c. Complejidad en tiempo (Notación O).

La complejidad en tiempo de BFS también es $O(b^d)$, ya que en el peor caso expande todos los nodos hasta alcanzar la profundidad donde se encuentra la meta. Como explora nivel por nivel sin priorizar caminos más prometedores, el número de expansiones crece exponencialmente conforme aumenta la profundidad de la solución.

d. Completitud

BFS es un algoritmo completo, lo que significa que garantiza encontrar una solución si esta existe, siempre que el espacio de estados sea finito. Esto ocurre porque explora sistemáticamente todos los estados alcanzables por niveles, evitando omitir posibles caminos hacia la meta.

e. Optimalidad

BFS es óptimo cuando todas las acciones tienen el mismo costo, ya que encuentra primero la solución con menor número de pasos. En este problema, como cada acción tiene costo uniforme, BFS garantiza retornar el plan más corto en cantidad de acciones.

f. Reflexión de la co-construcción con la IA

Durante el desarrollo de esta actividad utilicé inteligencia artificial como una herramienta de apoyo para revisar y corregir soluciones iniciales, así como para fortalecer la redacción de algunas secciones del informe. Aunque su uso fue necesario para contrastar ideas y detectar posibles errores, considero que se trató de una colaboración complementaria y no de una sustitución del proceso de aprendizaje. El análisis, la implementación y la comprensión del algoritmo fueron desarrollados de manera activa, utilizando la IA como un recurso de acompañamiento para refinar el trabajo realizado.

3. Búsqueda Regresiva (Backward Planning)

a. Complejidad en espacio (Notación O).

La complejidad en espacio del backwardSearch es $O(b^d)$, donde b es el factor de ramificación sobre el espacio de subobjetivos y d la profundidad del plan. A diferencia del forward BFS donde el estado es un conjunto de flujos concretos, aquí cada nodo es una descripción parcial del mundo (un subconjunto de flujos que deben ser verdaderos), lo que hace que el número de nodos distintos posibles sea exponencialmente mayor. Adicionalmente, se almacena el conjunto visited de subobjetivos ya explorados y el índice fluent_to_actions, ambos con un costo espacial adicional de $O(a \cdot n)$.

b. Complejidad en tiempo (Notación O).

La complejidad en tiempo también es $O(b^d)$ en el peor caso. Sin embargo, en la práctica el espacio de subobjetivos crece mucho más rápido que el espacio de estados forward, porque cada regresión a través de acciones como Move introduce nuevos flujos (Free, Adjacent) que expanden el subobjetivo. Esto se evidenció empíricamente: en tinyBase el backwardSearch expandió 29.328 estados en 5.99 segundos, frente a los 450 estados y 0.03 segundos del forwardBFS sobre el mismo layout

c. Completitud

El algoritmo es completo siempre que el espacio de subobjetivos sea finito y se evite visitar subobjetivos ya explorados. Dado que se utiliza un conjunto visited que descarta subobjetivos repetidos, y que las podas aplicadas (predicados estáticos imposibles y mutex) son sound (nunca eliminan soluciones válidas), el algoritmo garantiza encontrar un plan si existe alguno.

d. Optimalidad

El backwardSearch implementado con BFS es óptimo en número de acciones cuando todos los costos son uniformes, ya que explora los subobjetivos en orden de

profundidad creciente y retorna el primer plan encontrado, que corresponde al de menor cantidad de acciones. Esto se confirmó en tinyBase, donde el plan retornado tiene 9 acciones, idéntico al plan óptimo de referencia.

e. Reflexión de la co-construcción con la IA

Para el desarrollo del punto 3 se utilizó IA como herramienta de apoyo para optimizar una versión inicial funcional. La primera versión implementaba correctamente la lógica de regresión y el BFS sobre subobjetivos, pero no escalaba en layouts más grandes porque iteraba sobre todas las acciones grounded en cada nodo y no descartaba subobjetivos internamente inconsistentes. La IA ayudó a identificar dos mejoras concretas: construir un índice de acciones por fluente para reducir el número de candidatos evaluados en cada expansión, y agregar podas de mutex que eliminan subobjetivos imposibles como el robot en dos celdas simultáneamente o HandsFree y Holding activos al mismo tiempo. Estas correcciones fueron comprensibles y fáciles de integrar al código propio, y el resultado final es completamente explicable en una sustentación.

4. Heurística de ignorar precondiciones

4.a.b: Demostración de Admisibilidad

Para demostrar que la heurística ignorePreconditions es admisible, tomamos como ejemplo el mapa tinyBase. En este escenario, el objetivo final es tener el estado Rescued(patient_0). En la realidad, lograr esto requiere una secuencia de 9 acciones (moverse, recoger suministros, prepararlos, moverse, recoger al paciente, etc.). Sin embargo, en el problema relajado de esta heurística (donde ignoramos por completo las precondiciones), el planificador se da cuenta de que la acción Rescue(robot, patient_0, loc) directamente agrega el fluente Rescued(patient_0) a su add_list. Por lo tanto, la heurística estimará que el costo para llegar a la meta es de 1 sola acción. Dado que el costo estimado (1) es siempre menor o igual al costo real (9), se demuestra empíricamente que la heurística nunca sobreestima el costo real, confirmando que es admisible.

```

PS C:\Users\mateo\OneDrive\Documents\IA\PhoenixOperation> python main.py -p SimpleRescueProblem -f aStarPlanner -h ignorePreconditions
-l tinyBase -q

=====
Operación Fénix - ISIS-1611 Inteligencia Artificial
=====
Layout: tinyBase (5x7)
Problema: SimpleRescueProblem
Pacientes: ['patient_0']
Suministros: ['supplies_0']
Puestos médicos: [(1, 2)]

=====
Planificador: aStarPlanner
Heurística: ignorePreconditions

Tiempo de planificación: 1.334s
Estados expandidos: 252
Longitud del plan: 9 acciones

Plan:
1. Move(robot, (1, 4), (1, 3))
2. PickUp(robot, supplies_0, (1, 3))
3. Move(robot, (1, 3), (1, 2))
4. SetupSupplies(robot, supplies_0, (1, 2))
5. Move(robot, (1, 2), (1, 1))
6. PickUp(robot, patient_0, (1, 1))
7. Move(robot, (1, 1), (1, 2))
8. PutDown(robot, patient_0, (1, 2))
9. Rescue(robot, patient_0, (1, 2))

Ejecutando plan...

¡Misión completada exitosamente!

```

4.b.c y 4.b.d: Comparación y Análisis Empírico

Al probar las implementaciones sobre el layout tinyBase (plan óptimo de 9 acciones), obtuvimos los siguientes resultados en cuanto a estados expandidos:

- A* sin heurística (Búsqueda Ciega / Null): 345 estados expandidos.
- A* con ignorePreconditions: 252 estados expandidos.
- A* con ignoreDeleteLists: 138 estados expandidos.

¿Cuál es más informativa? La heurística de ignorar listas de borrado (ignoreDeleteLists) es considerablemente más informativa. La evidencia empírica lo respalda al haber expandido solo 138 estados frente a los 252 de la otra heurística. Esto ocurre porque ignoreDeleteLists sí tiene en cuenta las precondiciones de las acciones (obligando al planificador a seguir un orden lógico para avanzar), pero asume un mundo monótono donde nunca se retrocede. Esto genera una estimación de costo mucho más cercana al costo real (cota inferior más ajustada) que ignorePreconditions, la cual subestima drásticamente el esfuerzo al permitir "teletransportaciones" lógicas.

```

PS C:\Users\mateo\OneDrive\Documentos\IA\PhoenixOperation> python main.py -p SimpleRescueProblem -f aStarPlanner -h ignoreDeleteLists -l tinyBase -q

=====
Operación Fénix - ISIS-1611 Inteligencia Artificial
=====
Layout: tinyBase (5x7)
Problema: SimpleRescueProblem
Pacientes: ['patient_0']
Suministros: ['supplies_0']
Puestos médicos: [(1, 2)]

=====
Planificador: aStarPlanner
Heurística: ignoreDeleteLists

Tiempo de planificación: 1.025s
Estados expandidos: 138
Longitud del plan: 9 acciones

Plan:
1. Move(robot, (1, 4), (1, 3))
2. Pickup(robot, supplies_0, (1, 3))
3. Move(robot, (1, 3), (1, 2))
4. SetupSupplies(robot, supplies_0, (1, 2))
5. Move(robot, (1, 2), (1, 1))
6. Pickup(robot, patient_0, (1, 1))
7. Move(robot, (1, 1), (1, 2))
8. PutDown(robot, patient_0, (1, 2))
9. Rescue(robot, patient_0, (1, 2))

Ejecutando plan...

¡Misión completada exitosamente!

PS C:\Users\mateo\OneDrive\Documentos\IA\PhoenixOperation> python main.py -p SimpleRescueProblem -f aStarPlanner -h null -l tinyBase -q

=====
Operación Fénix - ISIS-1611 Inteligencia Artificial
=====
Layout: tinyBase (5x7)
Problema: SimpleRescueProblem
Pacientes: ['patient_0']
Suministros: ['supplies_0']
Puestos médicos: [(1, 2)]

=====
Planificador: aStarPlanner
Heurística: null

Tiempo de planificación: 0.037s
Estados expandidos: 345
Longitud del plan: 9 acciones

Plan:
1. Move(robot, (1, 4), (1, 3))
2. Pickup(robot, supplies_0, (1, 3))
3. Move(robot, (1, 3), (1, 2))
4. SetupSupplies(robot, supplies_0, (1, 2))
5. Move(robot, (1, 2), (1, 1))
6. Pickup(robot, patient_0, (1, 1))
7. Move(robot, (1, 1), (1, 2))
8. PutDown(robot, patient_0, (1, 2))
9. Rescue(robot, patient_0, (1, 2))

Ejecutando plan...

¡Misión completada exitosamente!

```

A continuación, se detalla el análisis del planificador A* y el cálculo de sus heurísticas relajadas en el contexto del problema de planificación de modo en que se entiende n = número de fuentes, a = número de acciones grounded, b = factor de ramificación, d = profundidad del plan óptimo.

- a. Complejidad en espacio (Notación O).
- Algoritmo A*: $O(b^d)$. A* es un algoritmo muy exigente en memoria, ya que debe mantener en la RAM todos los nodos generados en la frontera y el diccionario de estados ya visitados para evitar ciclos.

- Cálculo Heurístico por cada nodo: ignorePreconditions requiere $O(a)$ para tener las acciones disponibles al calcular la cobertura de conjuntos. ignoreDeleteLists requiere $O(n + a)$ para simular el crecimiento monótono del estado.

b. Complejidad en tiempo (Notación O).

- Algoritmo A*: En el peor de los casos, la complejidad temporal es $O(b^d)$ equivalente a una búsqueda ciega si la heurística no informa nada. Sin embargo, al usar nuestras heurísticas relajadas, el tiempo efectivo de expansión de nodos se reduce significativamente.
- Cálculo Heurístico por cada nodo: ignorePreconditions toma un tiempo de $O(|g| * a)$ evaluando iterativamente qué acción cubre la mayor cantidad del objetivo g . Por su parte, ignoreDeleteLists toma $O(k * a)$, siendo k los pasos necesarios en el problema relajado mediante Hill-Climbing.

c. Completitud

Sí, el algoritmo es completo. Dado que el costo de todas las acciones en el dominio es estrictamente positivo con costo = 1 y el espacio de estados es finito al usar el filtro de estados repetidos, A* garantiza encontrar una secuencia de acciones hacia el objetivo si esta existe.

d. Optimalidad

Sí, garantiza encontrar el plan de menor costo. Esta condición de optimalidad se garantiza estrictamente porque la búsqueda A* está siendo guiada por heurísticas que cumplen con la propiedad de admisibilidad.

- e. ¿Las heurísticas son admisibles? ¿Son consistentes? ¿Cuál es la ventaja del planificador HTN sobre el planificador clásico en términos de complejidad?

Ambas heurísticas nunca sobreestiman el costo real. La heurística ignorePreconditions relaja el problema asumiendo que cualquier acción puede ejecutarse al instante. La heurística ignoreDeleteLists relaja el problema impidiendo que los flujos se borren al ejecutar acciones. El costo de resolver un problema relajado siempre será menor o igual al costo de resolver el problema original.

Además de esto, satisfacen la desigualdad triangular. En ambos problemas relajados, dar un paso real (ejecutar una acción que cuesta 1) no puede hacer que nuestra

estimación de la distancia restante hacia la meta se reduzca mágicamente en más de 1 unidad.

f. Reflexión de la co-construcción con la IA

En cuanto a la integración de las herramientas de IA como apoyo en la codificación, se utilizó principalmente la extensión de Copilot en VS Code para completar la lógica de las heurísticas de los puntos 4a y 4b, lo cual facilitó la implementación técnica de las operaciones de conjuntos y la simulación del problema relajado. En cuanto al algoritmo A*, se utilizó la IA para corregir y refinar una implementación base cuya teoría ya estaba clara desde la clase, permitiendo ajustar detalles críticos de eficiencia como la gestión de estados repetidos mediante un diccionario de costos. Este proceso de co-construcción resultó en una solución final que es totalmente comprensible, lógica y fácil de sustentar, demostrando que la IA actuó como un soporte para mejorar la calidad del código y la eficiencia de la búsqueda sin comprometer la comprensión profunda de los fundamentos de la planificación.

5. Planificación Jerárquica de Tareas (HTN)

a. Complejidad en espacio (Notación O).

El algoritmo HTN implementado mediante hierarchicalSearch utiliza BFS sobre refinamientos jerárquicos, si b representa el número promedio de refinamientos posibles por HLA y d la profundidad máxima de refinamiento, la complejidad espacial es aproximadamente $O(b^d)$, ya que BFS debe almacenar simultáneamente todos los planes parciales generados. Además, cada plan puede contener múltiples HLAs y acciones primitivas, aumentando el consumo de memoria conforme crece la profundidad del refinamiento.

b. Complejidad en tiempo (Notación O).

En el peor caso, el algoritmo HTN explora todas las combinaciones posibles de refinamientos jerárquicos antes de encontrar una solución válida, por lo que su complejidad temporal también es aproximadamente $O(b^d)$. Adicionalmente, cada vez que se obtiene un plan completamente primitivo, este debe simularse acción por acción usando `apply_action` e `is_applicable`, agregando un costo adicional lineal en la longitud del plan.

c. Completitud

El algoritmo HTN implementado es completo siempre que todos los refinamientos posibles estén definidos y el espacio de refinamiento sea finito. Debido a que se utiliza BFS sobre planes jerárquicos, eventualmente se exploran todas las posibles descomposiciones de las HLAs, garantizando encontrar un plan válido si existe alguno.

d. Optimalidad

El HTN implementado no garantiza optimalidad global, aunque BFS encuentra la primera refinación válida, esto no implica necesariamente que sea el plan de menor costo, la optimalidad solo estaría garantizada si todos los refinamientos tuvieran el mismo costo y la búsqueda se realizara estrictamente en orden de costo acumulado. En la práctica, HTN prioriza reducir el espacio de búsqueda utilizando conocimiento del dominio antes que encontrar el plan absolutamente óptimo.

- e. ¿Las heurísticas son admisibles? ¿Son consistentes? ¿Cuál es la ventaja del planificador HTN sobre el planificador clásico en términos de complejidad?

En el punto 5, el planificador HTN no utiliza heurísticas como A* o Greedy Search, por lo que no se puede hablar directamente de heurísticas admisibles o consistentes. La planificación HTN se basa en descomponer tareas de alto nivel en subtarear más pequeñas mediante refinamientos definidos previamente, en lugar de estimar costos restantes hacia la meta, por esta razón, no aplica formalmente el análisis de admisibilidad ni consistencia. La principal ventaja del planificador HTN frente al planificador clásico es que reduce significativamente la complejidad del espacio de búsqueda, ya que no explora todas las acciones posibles en cada estado, sino únicamente aquellas definidas dentro de las jerarquías y refinamientos del dominio. Esto disminuye el número de estados expandidos y el tiempo de ejecución, especialmente en layouts grandes o complejos.

- f. Reflexión de la co-construcción con la IA

La IA fue utilizada principalmente como apoyo para depuración, comprensión de errores y refactorización de código, la primera versión de las funciones fue desarrollada manualmente, posteriormente, la IA ayudó a identificar problemas relacionados con refinamientos jerárquicos, generación de movimientos y validación de acciones, uno de los principales aprendizajes fue entender mejor cómo pequeñas inconsistencias en estructuras de datos o refinamientos podían impedir completamente la planificación. En general, las soluciones propuestas por la IA fueron comprensibles y útiles para fortalecer la comprensión de los conceptos de HTN.